Uma Revisão de Literatura sobre *Abordagens para Seleção e Priorização de Testes de Regressão para Ambientes Python*

Letícia Costa Oliveira Almeida

[1]Departamento de Ciências Exatas e da Terra, Campus I
Universidade do Estado da Bahia (UNEB)
Salvador, Bahia, Brasil.

leticiacostaoa@gmail.com

Resumo. *Neste trabalho será abordada uma revisão sistemática sobre as abordagens usadas para a seleção e priorização de testes de regressão, com foco em ambientes python que utilizem o pytest como framework de testes. A revisão seguiu um protocolo sistemático estruturado com base no framework PICOC, incluindo definição de questões de pesquisa, palavras-chave, critérios de busca, inclusão e exclusão, além de pontos mapeados para a extração das informações mais relevantes para responder o tema escolhido. Os resultados indicam predominância de técnicas baseadas em histórico de falhas, tempo de execução e mudanças na base de código, sendo que abordagens híbridas que combinam esses critérios demonstraram desempenho superior a técnicas isoladas, inclusive algumas baseadas em algoritmos de inteligência artificial. Conclui-se que, embora existam abordagens promissoras, ainda há lacunas na validação prática dessas técnicas, principalmente no que diz respeito à unificação de técnicas em uma única ferramenta e à validação em projetos python de código aberto.*

Abstract. *This paper presents a systematic literature review on approaches used for regression test selection and prioritization, focusing on Python environments that use pytest as the testing framework. The review followed a structured protocol based on the PICOC framework, including the definition of research questions, keywords, search criteria, inclusion and exclusion criteria, and data extraction points designed to answer the chosen topic. The results indicate a predominance of techniques based on failure history, execution time, and changes in the code base, while hybrid approaches that combine these criteria showed better performance than isolated techniques, including some based on artificial intelligence algorithms. It is concluded that, although promising approaches exist, there are still gaps in the practical validation of these techniques, especially regarding the integration of multiple techniques into a single tool and their validation in open source Python projects.*

1. Introdução

No contexto da engenharia de software, os testes constituem uma das principais estratégias para a garantia da qualidade do mesmo durante seu ciclo de vida. Conforme o padrão IEEE número 24765:2017, testes são definidos como o processo de avaliação de um sistema ou componente com o objetivo de verificar se ele satisfaz os requisitos especificados e identificar diferenças entre o comportamento esperado e o observado [801 2017].

Processos de construção de software modernos recomendam que mudanças sejam frequentemente integradas à linha principal de desenvolvimento do projeto. Antes disso, é necessário garantir que as alterações mais recentes não impeçam o funcionamento de algum módulo do sistema, o que pode ser feito com os testes de regressão. Contudo, conforme a base de código evolui e novos testes são adicionados, a suíte de regressão tende a crescer continuamente, tornando sua execução integral cada vez mais custosa [Celik et al. 2017].

Nesse cenário, fluxos de Integração Contínua, ou CI (abreviado do inglês *"Continuous Integration"*) surgem como uma solução para automatizar a verificação do código a cada nova mudança, executando os testes antes que alterações sejam incorporadas ao projeto. Porém, executar a suíte completa a cada alteração torna-se inviável quando ela cresce demais, gerando um tempo de feedback muito longo [Elsner et al. 2022]. Em um estudo de Cheng et al. [Cheng et al. 2024] identificaram suítes reais com duração média de até 6,5 horas por execução completa, tempo incompatível com ciclos ágeis de desenvolvimento. Assim, surge a necessidade de técnicas de otimização da suíte de execução dos testes, como a priorização e a seleção, que buscam, respectivamente, reordenar os testes para detectar falhas o mais cedo possível, e selecionar um subconjunto relevante para cada ciclo de execução [Chen et al. 2018, Celik et al. 2017].

No ecossistema Python, o pytest consolidou-se como o principal framework de testes automatizados em projetos de código aberto [Cheng et al. 2025]. Plugins (componentes que estendem ou modificam as funcionalidades de um sistema principal, sem exigir alterações na sua base de código) como o pytest-rts [Kauhanen et al. 2021] e o pytest-ranking [Cheng et al. 2025] representam os avanços nesse ambiente, oferecendo, respectivamente, seleção e priorização de testes integradas ao pytest. Contudo, nenhuma delas combina as duas estratégias em um único mecanismo, o que limita os ganhos possíveis em termos de redução do tempo de execução. Logo, esta revisão sistemática de literatura (RSL) tem como objetivo mapear e analisar métodos e técnicas já estudados, buscando fundamentos para a construção de uma solução integrada de seleção e priorização de testes de regressão para o ecossistema Python/pytest.

2. Planejamento da Revisão de Literatura

2.1. Questões de Pesquisa

Seguindo o contexto do objetivo desta RSL, a questão principal de pesquisa é: ***Quais técnicas e ferramentas de seleção e priorização de testes de regressão são aplicáveis ao ecossistema Python, com o pytest (framework de testes em python) e contribuem para a redução do tempo de execução da suíte sem comprometer a detecção de defeitos?***. As questões secundárias de pesquisa para esta revisão concentram-se em:

- Quais os contextos e tipos de projeto em que as técnicas de seleção e/ou priorização estão sendo empregadas?
- Quais técnicas de seleção de testes de regressão não baseadas em inteligência artificial (IA) vêm sendo aplicadas hoje em dia?
- Quais são as métricas para avaliação dos métodos de seleção e priorização de testes?

2.2. Framework PICOC

O framework PICOC foi selecionado para refletir o problema de pesquisa e criar uma metodologia de busca cirúrgica. As 5 dimensões foram definidas como:

- **População (*Population*)**: Sistemas de software que utilizam testes de regressão automatizados, especialmente no ambiente Python e de preferência o pytest como framework de testes automatizados;
- **Intervenção (*Intervention*)**: Técnicas e ferramentas de priorização e seleção de testes de regressão;
- **Comparação (*Comparison*)**: Outros plugins e técnicas de priorização e/ou seleção de testes de regressão, além da execução integral da suíte de testes crua na ordem padrão do pytest;
- **Resultado (*Outcome*)**: Redução do tempo de execução da suíte de testes mantendo a eficácia na detecção de defeitos;
- **Contexto (*Context*)**: Fluxos de CI, em projetos Python de código aberto disponíveis no GitHub.

2.3. Palavras Chave

As palavras-chave foram escolhidas a partir da decomposição do framework PICOC, que serve como um mapeamento para os tópicos a serem pesquisados na RSL. São elas:

- "Test Selection";
- "Test Prioritization";
- "Regression Testing";
- "Python";
- "Pytest";
- "Open Source".

2.4. Bases de Dados

Para realizar a busca, foram escolhidos o Springer, o ACM Digital Library e o Google Acadêmico como repositórios. Os 2 primeiros apresentam um acervo rico em produções de engenharia e qualidade de software, com periódicos técnicos internacionais, já o terceiro foi incluído como base complementar em razão do número reduzido de estudos recuperados nas buscas iniciais nas outras duas bases, visando garantir maior abrangência na cobertura da literatura disponível sobre o tema. Os parâmetros de busca considerados foram uma janela temporal de 10 anos (2016-2026) e o inglês como idioma das produções.

2.5. String de Busca

Nesse sentido, a string de busca foi construída por meio das palavras-chave e dos operadores lógicos “AND”, “OR” e "NOT", com variações de sintaxe seguindo a lógica de determinados repositórios, subdividindo a mesma em duas versões.

- **ACM Digital Library e Springer Link:** ("test selection"OR "test case selection"OR "regression test selection"OR "test prioritization"OR "test case prioritization") AND ("regression testing") AND ("python") AND NOT ("test generation"OR "artificial intelligence")
  Retorno: 54 artigos do ACM Digital Library e 39 do Springer Link.

- **Google Acadêmico:** ("test selection"OR "test case selection"OR "regression test selection"OR "test prioritization"OR "test case prioritization") "regression testing"“python” “pytest” -"test generation-"artificial intelligence"
  Retorno: 32 artigos.
  Removendo Duplicados: 28 artigos.

Além da diferença de sintaxe em relação às strings usadas nos outros dois repositórios, a string do Google Acadêmico inclui *“pytest”* como termo adicional, permitindo filtrar melhor os resultados em uma base que, por ser mais abrangente em volume de publicações, exige parâmetros de busca mais específicos.

2.6. Critérios de Inclusão e Exclusão

Os critérios de inclusão e exclusão foram definidos visando selecionar estudos que melhor se encaixavam no tema da pesquisa. Um artigo será automaticamente descartado se apresentar pelo menos um dos critérios de exclusão, e o mesmo vale para os de inclusão. São eles:

- **Inclusão:**
  1. Estudos inseridos no contexto de testes de regressão;
  2. Estudos com métodos, técnicas ou ferramentas aplicáveis à seleção/priorização de testes;
  3. Estudos com possibilidade de aplicação em projetos open source (de código aberto); e
  4. Estudos que usam Python/pytest ou abordagens adaptáveis a esse ambiente.

- **Exclusão:**
  1. Estudos com foco exclusivo em testes manuais;
  2. Estudos com foco exclusivo em testes de configuração;
  3. Estudos com foco exclusivo na geração de testes;
  4. Estudos com foco exclusivo em flaky tests;
  5. Estudos baseados exclusivamente em IA ou Machine Learning (ML - Aprendizado de Máquina) na seleção e priorização de testes;
  6. Estudos do tipo survey ou que não estejam em revista indexada;
  7. Estudos cujo texto completo não esteja disponível;
  8. Estudos em idiomas diferentes de inglês;
  9. Estudos cujo método não seja aplicável para seleção ou priorização;
  10. Estudos cujo método não seja aplicável ao ambiente python com pytest.

2.7. Formulário de Extração de Dados

Para sistematizar a análise dos artigos selecionados na triagem inicial, foi elaborado um formulário de extração dos dados em formato de planilha, com campos definidos a partir das questões de pesquisa e das dimensões do framework PICOC, visando facilitar responder às perguntas de pesquisa. Os principais dados mapeados em cada estudo foram:

- Metadados bibliográficos (título, autores, ano de publicação, buscador, URL e DOI);

- Tipo da técnica proposta (Priorização, seleção, ambas ou outra);
- Linguagem base (Python, Java, C/C++, JavaScript, Multilinguagem ou outra);
- Ferramenta desenvolvida;
- Critério base para técnica (mudanças no código, falhas, cobertura de código, tempo de execução, features, relação entre componentes do sistema);
- Algoritmo ou método proposto;
- Contexto e dataset de avaliação;
- Métrica de avaliação utilizada; e
- Limitações encontradas.

2.8. Condução

A condução desta revisão sistemática foi planejada em duas fases sequenciais, aplicando os critérios de inclusão e exclusão definidos na Seção 2.6 de forma progressiva sobre o conjunto de artigos encontrados. Na primeira fase, são coletados os resultados das três bases de dados (ACM Digital Library, Springer Link e Google Acadêmico) a partir das strings de busca descritas na Seção 2.5. A partir desse retorno, os artigos duplicados são removidos manualmente, e os restantes passam por uma etapa de triagem inicial.

Na segunda fase, a triagem inicial ocorre por meio da leitura de títulos e resumos, aplicando os critérios de inclusão e exclusão. Publicações que não apresentassem informações suficientes no título ou resumo para uma decisão clara foram mantidas para leitura completa. Ao final dessa etapa, os artigos não removidos pela triagem são lidos na íntegra, visando também responder os pontos presentes no formulário de extração de dados descrito na Seção 2.7. Nessa etapa, os critérios de inclusão e exclusão foram aplicados com maior rigor, resultando em produções incluídas para síntese final dos resultados. O processo completo está ilustrado no fluxograma PRISMA 2020, apresentado na Seção 3.

3. Execução da Revisão Sistemática

Ao total, com a remoção das quatro duplicatas, as 3 bases retornaram 121 (cento e vinte e um) artigos, os quais precisaram passar pela triagem inicial, que engloba a leitura do título e do resumo, aplicando os critérios de inclusão e exclusão. Como resultado, sobraram 25 (vinte e cinco) produções distribuídas entre as fontes de busca, como pode ser visto no gráfico abaixo:

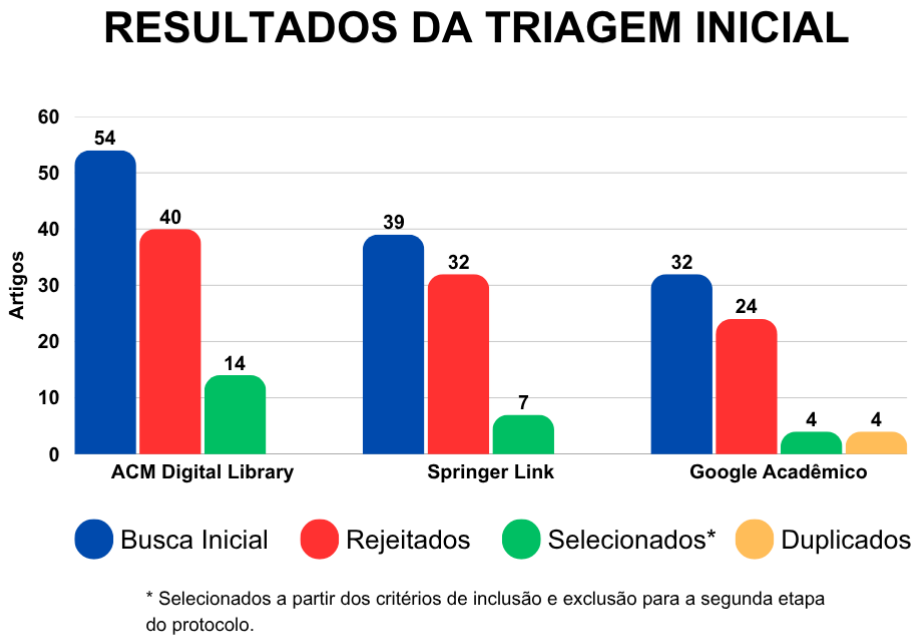


Figura 1. Resultados da triagem inicial da revisão sistemática.

Assim, os 25 artigos foram novamente avaliados por meio dos critérios de inclusão e exclusão, mas desta vez lidos na íntegra, visando extrair os dados requisitados na planilha de extração, elegendo desta forma, 11 produções acadêmicas finais. O processo completo de triagem, desde a busca nas bases até a seleção final, está representado no fluxograma PRISMA 2020 [PRISMA Statement 2020], conforme ilustrado na Figura 2.

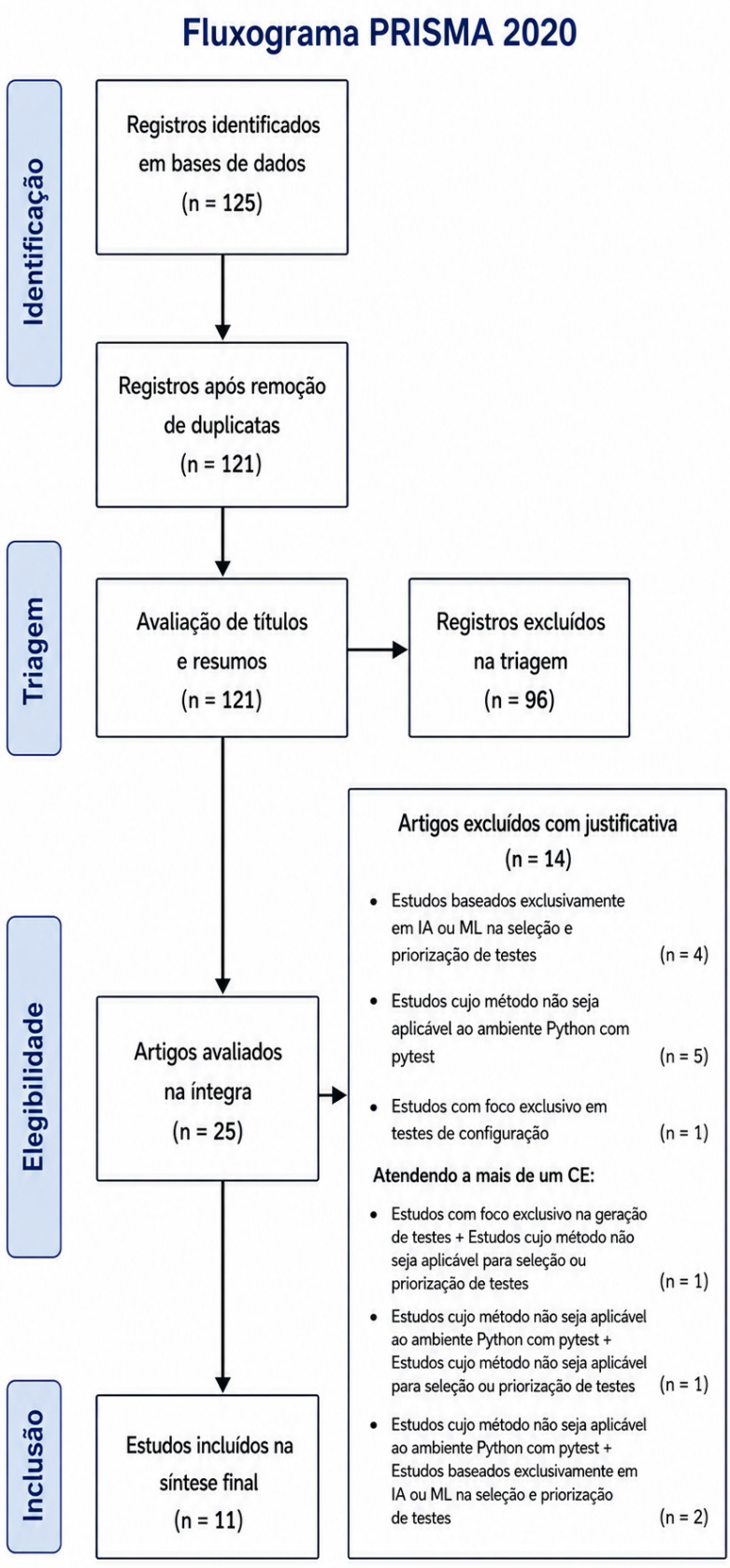


Figura 2. Fluxograma PRISMA 2020 da seleção de estudos.

4. Síntese dos Resultados

A partir da leitura e extração de informações dos artigos selecionados, os principais pontos mapeados que respondem as questões secundárias desta RSL foram:

- **Técnicas simples superam abordagens complexas em CI**
  Foi observado consistentemente em alguns artigos que critérios simples baseados em histórico e *runtime* (tempo de execução) igualam ou até mesmo superam abordagens mais complexas como técnicas baseadas em ML em contextos de CI, especialmente quando avaliadas em cenários realistas [Cheng et al. 2025]. Cheng et al. demonstraM, avaliando 59 técnicas em suítes de longa duração, que abordagens híbridas simples, combinando tempo de execução e histórico de falhas, por exemplo, foram as mais eficazes [Cheng et al. 2024]. Já Elsner et al. mostram que obteve redução média de 84% no tempo de execução mantendo detecção de 90% das falhas usando apenas metadados de CI e Git, sem acesso ao código-fonte [Elsner et al. 2021].
- **Priorização e seleção como técnicas complementares**
  Dentro dos artigos incluídos, observa-se que executar seleção antes da priorização amplifica os resultados: a seleção reduz o conjunto de testes a executar, e a priorização reordena os selecionados para detecção mais rápida [Raamesh et al. 2022, Kumari et al. 2025, Kauhanen et al. 2021, Elsner et al. 2021].

  Apesar do fato evidenciado acima, também foi notada a ausência de ferramentas deste tipo para o python e o pytest. Dos 11 artigos incluídos, apenas pytest-ranking, [Cheng et al. 2025] e pytest-rts [Kauhanen et al. 2021] oferecem ferramentas diretamente integradas a esse ecossistema, mas nenhuma delas combina as duas estratégias em um único plugin.

  Outras referências trazem algoritmos como o HBR2O (*Hybrid Battle Royale-based Remora Optimization*) [Raamesh et al. 2022] e o GWO_TCO (*Grey Wolf Optimization for Test Case Optimization*) [Kumari et al. 2025], que também implementam ambas as técnicas mas foram avaliados em Java/C++. O HBR2O é a junção dos algoritmos Battle Royale Optimization (BRO), que fica responsável por eliminar testes redundantes e garantir que áreas diferentes do sistema sejam testadas, com o Remora Optimization (RO), que prioriza os testes selecionados pelo BRO para economizar recursos e detectar falhas o mais rápido possível. Já o GWO_TCO, usa a priorização para encontrar o melhor caminho de testes, com base naqueles que detectam mais falhas em menos tempo, e a seleção é aplicada posteriormente: verificando se as falhas que um caso de teste detecta já foram expostas por outros que vieram antes na ordem de prioridade, sendo assim eliminado por redundância.
- **Principais critérios utilizados para a seleção ou priorização**
  Artigos mostram que o rastreamento de mudanças de código é o critério mais preciso para seleção, pois vincula diretamente testes a alterações no código-base [Araújo et al. 2017, Kauhanen et al. 2021]. O pytest-rts [Kauhanen et al. 2021] implementa isso via cobertura e análise de diferenças com o Git, mas com limitações conhecidas de imprecisão em decoradores e importações Python. Já Elsner et al. reforça que mesmo sem acesso ao código-fonte, metadados de

mudanças, como os nomes de arquivos alterados, já são suficientes para ganhos significativos [Elsner et al. 2021].

Em muitas produções também foi identificada a exploração das técnicas de priorização ou seleção baseada em tempo de execução combinado a um segundo fator além de mudanças em código, como histórico de falhas [Garg and Shekhar 2024, Khanna et al. 2019, Cheng et al. 2025] e cobertura de código [Kumari et al. 2025, Kauhanen et al. 2021]. Casos mais isolados ainda trazem mapeamento de features [Mendonça et al. 2023] e relação entre componentes do sistema [Torbunova et al. 2024], respectivamente, como parâmetro para seleção e priorização de casos de teste.

- **Principais contextos e sistemas de aplicação das técnicas**
  Em relação aos contextos de aplicação, foi observada uma predominância de avaliações em projetos Java, com uso de datasets comparativos consolidados na área, como o SIR (*Software Infrastructure Repository*) e o Defects4J [Garg and Shekhar 2024, Kumari et al. 2025, Raamesh et al. 2022]. Apenas três dos onze artigos incluídos foram avaliados nativamente em projetos Python: o pytest-ranking [Cheng et al. 2025], avaliado em projetos open source do GitHub com pipelines de CI; o pytest-rts [Kauhanen et al. 2021], testado em três projetos Python reais (Flask, Rich e python-rsa); e o trabalho de Torbunova et al. que implementou a ferramenta em Python, mas a aplicou sobre datasets industriais [Torbunova et al. 2024]. Os demais artigos foram avaliados em ambientes multi-linguagem. Quanto aos ambientes de aplicação, também nota-se uma divisão entre estudos conduzidos em sistemas industriais reais [Elsner et al. 2021, Torbunova et al. 2024] e em projetos open source, como argumentado anteriormente.
- **Principais métricas de avaliação**
  Entre os 11 artigos selecionados, as métricas mais recorrentes para avaliação dos métodos de priorização e seleção foram o APFD (*Average Percentage of Faults Detected* — Percentual Médio de Falhas Detectadas) e o APFDc (*Average Percentage of Faults Detected considering costs* — Percentual Médio de Falhas Detectadas por Custo), além do tempo total de execução da suíte otimizada e do percentual de redução no número de testes executados.

  O APFD mede a velocidade com que uma determinada ordem de execução de testes consegue detectar falhas ao longo da execução da suíte, o valor de medição varia de 0 a 1, quanto maior o valor, mais cedo as falhas são encontradas. Já o APFDc funciona como uma extensão da APFD, incorporando também o custo de execução de cada caso de teste e a severidade das falhas que ele detecta, tornando a avaliação mais realista para cenários onde os testes possuem durações e importâncias distintas. Apesar de ser utilizado em muitas produções, o APFD pode ser enganoso em suítes de longa duração, pois não reflete diretamente a economia de tempo obtida — um aspecto central no contexto de CI, onde o objetivo é reduzir o tempo total de execução e não apenas reordenar os testes [Cheng et al. 2024].

  De forma mais pontual, algumas outras referências adotam métricas de avaliação complementares a seu contexto, como por exemplo:

- Porcentagem de requisitos cobertos e cobertura de dependências [Raamesh et al. 2022];
- Taxa de severidade identificadas [Khanna et al. 2019];
- Taxa de falhas omitidas na execução da suíte [Kauhanen et al. 2021]; e
- Complexidade do código e histórico do teste, combinados em um índice de sensibilidade à falhas criado pelos autores em [Garg and Shekhar 2024].

5. Conclusões

Após a análise e interpretação das informações extraídas da planilha, fica evidente que existem diversos critérios, contextos e métricas de avaliação para técnicas de priorização e seleção de testes de regressão. Entre eles, as abordagens melhor avaliadas incluem utilização do histórico de mudanças e cobertura de código, combinados a tempo de execução da suíte e taxa de detecção de falhas como medidas de validação.

Por outro lado, nota-se que vários artigos possuem limitações em comum, como a ausência de aplicação das técnicas em softwares de código aberto em python com o pytest como framework de testes, principalmente considerando a aplicação híbrida da priorização e seleção dos testes de regressão. Ademais, a predominância do APFD como métrica única não captura adequadamente o objetivo de redução de tempo de execução e em suítes com muitos testes rápidos, o APFD pode mascarar ganhos reais. Apesar de o APFDc ser adotado em algumas publicações, não é notada uma variabilidade de métricas de avaliação: poucos artigos usam o percentual de redução da suíte ou o tempo até a primeira falha (TTFF) como complementares.

A partir dessas observações, as lacunas identificadas nesta RSL serão endereçadas em um trabalho de conclusão de curso por meio do desenvolvimento de um plugin para o pytest que unifique as estratégias de seleção e priorização de testes de regressão em uma única ferramenta integrada ao ecossistema Python. A proposta parte do pytest-ranking como base de priorização já consolidada [Cheng et al. 2025] e busca complementá-lo com uma camada de seleção de testes baseada em mudanças na base de código, avaliando a abordagem em projetos Python open source disponíveis no GitHub. Dessa forma, pretende-se contribuir tanto com uma ferramenta prática para o ambiente Python quanto com evidências sobre a eficácia da combinação de seleção e priorização para identificação de falhas de forma otimizada em um contexto ainda pouco explorado na literatura.

Referências

(2017). Iso/iec/ieee international standard - systems and software engineering–vocabulary. *ISO/IEC/IEEE 24765:2017(E)*, pages 1–541.

Araújo, J., Araújo, J., Magalhães, C., Andrade, J., and Mota, A. (2017). Feasibility of using Source Code Changes on the Selection of Text-based Regression Test Cases. In *Proceedings of the 2nd Brazilian Symposium on Systematic and Automated Software Testing*, SAST '17, New York, NY, USA. Association for Computing Machinery.

Biswas, S., Rathi, R., Dutta, A., Mitra, P., and Mall, R. (2021). A regression test case prioritization technique targeting 'hard to detect' faults. *International Journal of System Assurance Engineering and Management*.

Celik, A., Vasic, M., Milicevic, A., and Gligoric, M. (2017). Regression test selection across JVM boundaries. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2017, pages 809–820, New York, NY, USA. Association for Computing Machinery.

Chen, J., Lou, Y., Zhang, L., Zhou, J., Wang, X., Hao, D., and Zhang, L. (2018). Optimizing test prioritization via test distribution analysis. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2018, pages 656–667, New York, NY, USA. Association for Computing Machinery.

Chen, Z., Chen, J., Wang, W., Zhou, J., Wang, M., Chen, X., Zhou, S., and Wang, J. (2023). Exploring Better Black-Box Test Case Prioritization via Log Analysis. *ACM Trans. Softw. Eng. Methodol.*, 32(3).

Cheng, R., Ke, K., and Marinov, D. (2025). pytest-ranking: A Regression Test Prioritization Tool for Python. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, FSE Companion '25, pages 1089–1093, New York, NY, USA. Association for Computing Machinery.

Cheng, R., Wang, S., Jabbarvand, R., and Marinov, D. (2024). Revisiting Test-Case Prioritization on Long-Running Test Suites. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2024, pages 615–627, New York, NY, USA. Association for Computing Machinery.

Cheng, R., Zhang, L., Marinov, D., and Xu, T. (2021). Test-case prioritization for configuration testing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2021, pages 452–465, New York, NY, USA. Association for Computing Machinery.

Elsner, D., Hauer, F., Pretschner, A., and Reimer, S. (2021). Empirically evaluating readily available information for regression test optimization in continuous integration. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2021, pages 491–504, New York, NY, USA. Association for Computing Machinery.

Elsner, D., Wuersching, R., Schnappinger, M., Pretschner, A., Graber, M., Dammer, R., and Reimer, S. (2022). Build system aware multi-language regression test selection in continuous integration. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, ICSE-SEIP '22, pages 87–96, New York, NY, USA. Association for Computing Machinery.

Garg, K. and Shekhar, S. (2024). Optimizing test case prioritization through ranked NSGA-2 for enhanced fault sensitivity analysis. *Innovations in Systems and Software Engineering*.

Hu, Y., Ishibe, H., Dai, F., Yamazaki, T., and Chiba, S. (2024). Bugfox: A Trace-Based Analyzer for Localizing the Cause of Software Regression in JavaScript. In *Proceedings of the 17th ACM SIGPLAN International Conference on Software Language Engineering*, SLE '24, pages 224–233, New York, NY, USA. Association for Computing Machinery.

Kauhanen, E., Nurminen, J. K., Mikkonen, T., and Pashkovskiy, M. (2021). Regression test selection tool for python in continuous integration process. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 618–621. IEEE.

Khanna, M., Chaudhary, A., Toofani, A., and Pawar, A. (2019). Performance Comparison of Multi-objective Algorithms for Test Case Prioritization During Web Application Testing. *Arabian Journal for Science and Engineering*.

Kumari, S., Jindal, S., and Sharma, A. (2025). Test case optimization using grey wolf algorithm. *Software Quality Journal*.

Mahdieh, M., Mirian-Hosseinabadi, S.-H., and Mahdieh, M. (2022). Test case prioritization using test case diversification and fault-proneness estimations. *Automated Software Engineering*.

Meier, D., Mattis, T., and Hirschfeld, R. (2021). Toward Exploratory Understanding of Software using Test Suites. In *Companion Proceedings of the 5th International Conference on the Art, Science, and Engineering of Programming*, Programming '21, pages 60–67, New York, NY, USA. Association for Computing Machinery.

Mendonça, W. D. F., Assunção, W. K. G., and Vergilio, S. R. (2023). Feature-oriented Test Case Selection during Evolution of Highly-Configurable Systems. In *Proceedings of the 27th ACM International Systems and Software Product Line Conference - Volume A*, SPLC '23, pages 76–86, New York, NY, USA. Association for Computing Machinery.

Mendonça, W. D. F., Vergilio, S. R., Michelon, G. K., Egyed, A., and Assunção, W. K. G. (2022). Test2Feature: feature-based test traceability tool for highly configurable software. In *Proceedings of the 26th ACM International Systems and Software Product Line Conference - Volume B*, SPLC '22, pages 62–65, New York, NY, USA. Association for Computing Machinery.

PRISMA Statement (2020). Prisma 2020 flow diagram. `https://www.prisma-statement.org/prisma-2020-flow-diagram`. Acesso em: 30 abr. 2026.

Raamesh, L., Radhika, S., and Jothi, S. (2022). A cost-effective test case selection and prioritization using hybrid battle royale-based remora optimization. *Neural Computing and Applications*.

Rosenbauer, L., Stein, A., and Hähner, J. (2022). A Germinal Center Artificial Immune System for Black Box Test Selection. *SN Computer Science*, 4(1):55.

Sakhrawi, Z. and Labidi, T. (2024). Test case selection and prioritization approach for automated regression testing using ontology and COSMIC measurement. *Automated Software Engineering*.

Taseen, R. L. (2023). Enhancing Efficiency of the Test Case Prioritization Technique by Improving the Rate of Fault Detection. Master's thesis, Lamar University-Beaumont.

Torbunova, A., Strandberg, P. E., and Porres, I. (2024). Dynamic Test Case Prioritization in Industrial Test Result Datasets. In *Proceedings of the 5th ACM/IEEE International*

Conference on Automation of Software Test (AST 2024), AST '24, pages 154–158, New York, NY, USA. Association for Computing Machinery.

Wang, H., Yi, P. L., Parladorio, J., Lam, W., Marinov, D., and Xie, T. (2024). Hierarchy-Aware Regression Test Prioritization. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, pages 343–354. IEEE.

6. Planilha-resumo de Resultados

Nesta seção, a planilha-resumo abrange os trabalhos selecionados após a triagem inicial para a seleção final. As colunas intermediárias representam os critérios de inclusão e exclusão da RSL. Dentre os dez parâmetros de exclusão, somente cinco são pontuados na tabela baixo, já que nenhum dos 25 artigos analisados se encaixou na outra metade dos testes de inclusão.

Tabela 1. Planilha-resumo dos trabalhos selecionados.

Identificação do Trabalho	I1	I2	I3	I4	E2	E3	E5	E9	E10	Descrição	Avaliação
Sakhrawi, Z. and Labidi, T. (2024). Test case selection and prioritization approach for automated regression testing using ontology and COSMIC measurement. *Automated Software Engineering*							X			Propõe seleção e priorização de casos de teste usando ontologia e similaridade semântica via WordNet, combinados à medição COSMIC para estimar o esforço de cada teste. Avaliado em dados industriais automotivos reais.	Excluído por uso de técnicas de PLN (WordNet/Sematch) como método central de seleção. Apesar de não ser ML supervisionado, o critério de exclusão E5 foi aplicado por ser baseado em processamento de linguagem natural.
Garg, K. and Shekhar, S. (2024). Optimizing test case prioritization through ranked NSGA-2 for enhanced fault sensitivity analysis. *Innovations in Systems and Software Engineering*	X	X		X						Propõe o uso do algoritmo evolutivo Ranked NSGA-2 para priorização de testes com múltiplos objetivos: índice de sensibilidade a falhas (criado pelos autores), custo de execução e APFD. Avaliado em oito aplicações Java (cinco criadas por estudantes e três do dataset SIR).	Incluído como referência de abordagem multi-objetivo de priorização sem ML. O Índice de Sensibilidade proposto é uma contribuição métrica relevante. Limitado pela avaliação exclusiva em Java e pela dependência de aplicações criadas por estudantes, o que pode introduzir viés.
Mahdieh, M., Mirian-Hosseinabadi, S.-H., and Mahdieh, M. (2022). Test case prioritization using test case diversification and fault-proneness estimations. *Automated Software Engineering*							X		X	Propõe o método CovClustering+FP, que combina clusterização hierárquica por cobertura com predição de propensão a falhas via XGBoost. Avaliado no dataset Defects4J com 357 versões de projetos Java.	Excluído pelo uso de ML supervisionado (XGBoost) como etapa central do método e pela avaliação ser exclusivamente em Java sem qualquer menção a Python ou pytest.
Raamesh, L., Radhika, S., and Jothi, S. (2022). A cost-effective test case selection and prioritization using hybrid battle royale-based remora optimization. *Neural Computing and Applications*	X	X	X	X						Propõe o algoritmo híbrido HBR2O (*Hybrid Battle Royale-based Remora Optimization*), que combina o BRO para eliminar testes redundantes via seleção e o RO para priorizar os testes selecionados. Implementado em Python e avaliado em cinco projetos. Métricas: AFD%, cobertura de sentenças, tamanho de memória e tempo de execução.	Incluído como referência de abordagem híbrida seleção+priorização. A implementação em Python é um diferencial. Limitação relevante é a complexidade quadrática do algoritmo e a necessidade de calibração manual dos pesos, o que pode dificultar a escalabilidade em projetos maiores.

Continua na próxima página

Tabela 1 – continuação da página anterior

Identificação do Trabalho	I1	I2	I3	I4	E2	E3	E5	E9	E10	Descrição	Avaliação
Kumari, S., Jindal, S., and Sharma, A. (2025). Test case optimization using grey wolf algorithm. *Software Quality Journal*	X	X	X	X						Propõe o GWO_TCO (*Grey Wolf Optimization for Test Case Optimization*), algoritmo metaheurístico inspirado no comportamento de caça dos lobos cinzentos, que combina priorização (melhor caminho de detecção de falhas) e seleção por redundância. Implementado via Python/Google Colab. Avaliado em 17 programas open source e no dataset SIR. Métricas: APFD, FC, ETE e PE.	Incluído como referência de abordagem híbrida seleção+priorização com conjunto diversificado de métricas. Redução de ≈80% no tamanho da suíte mantendo APFD alto. Avaliado em Java/C++, sem integração direta com pytest.
Biswas, S., Rathi, R., Dutta, A., Mitra, P., and Mall, R. (2021). A regression test case prioritization technique targeting 'hard to detect' faults. *International Journal of System Assurance Engineering and Management*									X	Propõe a técnica HDFDC, que usa testes de mutação para construir uma matriz de falhas e priorizar testes que detectam falhas de difícil detecção (*hard to detect faults*). Avaliado em 22 projetos Java dos datasets Defects4J e SIR. Métrica: APFD e APFDc.	Excluído por inviabilidade de aplicação no ambiente Python/pytest. O método exige mutation testing como pré-requisito obrigatório, gerando custo de execução incompatível com o objetivo de reduzir o tempo de execução da suíte. Sem menção a Python ou pytest.
Khanna, M., Chaudhary, A., Toofani, A., and Pawar, A. (2019). Performance Comparison of Multi-objective Algorithms for Test Case Prioritization During Web Application Testing. *Arabian Journal for Science and Engineering*		X	X							Compara os algoritmos NSGA-II, vNSABC e Programação Dinâmica para priorização de testes em aplicações web. O vNSABC superou os demais em velocidade e eficácia. Código Python da abordagem de PD disponível no apêndice. Métricas: APFDc, iterações, tempo de execução e taxa de severidade.	Incluído como referência de comparação de algoritmos multi-objetivo de priorização. O código Python disponível e o uso do APFDc como métrica são pontos positivos. Avaliado em websites JSP, sem integração com pytest.
Kauhanen, E., Nurminen, J. K., Mikkonen, T., and Pashkovskiy, M. (2021). Regression test selection tool for python in continuous integration process. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 618–621. IEEE	X	X	X	X						Apresenta o pytest-rts, plugin para pytest que realiza seleção de testes por cobertura de código e mudanças via git diff, com priorização por menor tempo de execução. Avaliado em Flask, Rich e python-rsa. Métrica: taxa de falhas omitidas e redução do conjunto de testes.	Incluído por trazer uma ferramenta Python/pytest open source que realiza seleção e priorização. Taxa de omissão de falhas de 16–24% na granularidade de linha é uma limitação importante que o trabalho proposto busca endereçar.
Wang, H., Yi, P. L., Parladorio, J., Lam, W., Marinov, D., and Xie, T. (2024). Hierarchy-Aware Regression Test Prioritization. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, pages 343–354. IEEE									X	Propõe meta-técnicas de priorização hierárquica (HA RTP) que adaptam técnicas existentes para considerar o custo de troca entre módulos Maven/JUnit. Introduz a métrica HAPFDc. Avaliado em datasets IRBRTP e RTPTorrent, todos em Java.	Excluído por inviabilidade de aplicação no ambiente Python/pytest. O custo de troca entre módulos (SATC) é específico da JVM e não tem equivalente direto no pytest. Avaliação exclusivamente em Java.

Continua na próxima página

Tabela 1 – continuação da página anterior

Identificação do Trabalho	I1	I2	I3	I4	E2	E3	E5	E9	E10	Descrição	Avaliação
Taseen, R. L. (2023). Enhancing Efficiency of the Test Case Prioritization Technique by Improving the Rate of Fault Detection. Master's thesis, Lamar University-Beaumont									X	Propõe o algoritmo Total_TCP, que calcula o ETCR (*Effective Test Case Ranking* ou Ranqueamento de casos de testes efetivo) como soma de quatro fatores para priorização. Implementado com QTP 9.x para projetos em Visual Basic. Métricas: APFD e percentual de casos necessários para detectar todas as falhas.	Excluído por ser restrito ao ambiente Visual Basic/QTP, sem possibilidade de simples adaptação ao ecossistema Python/pytest. Escopo de avaliação muito limitado (dois projetos pequenos).
Rosenbauer, L., Stein, A., and Hähner, J. (2022). A Germinal Center Artificial Immune System for Black Box Test Selection. *SN Computer Science*, 4(1):55							X			Apresenta o GCAIS (*Germinal Center Artificial Immune System* ou), heurística evolutiva multi-objetivo inspirada no sistema imunológico para seleção de testes em modo black box. Implementado com interface gráfica e entregue à BSH Home Appliances. Avaliado em datasets industriais automotivos.	Excluído por ser uma metaheurística evolutiva aplicada a testes de sistema industriais em modo black box, sem integração com frameworks de teste de código. O contexto de testes manuais/semi-automatizados de eletrodomésticos é incompatível com pytest.
Cheng, R., Ke, K., and Marinov, D. (2025). pytest-ranking: A Regression Test Prioritization Tool for Python. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, FSE Companion '25, pages 1089–1093, New York, NY, USA. Association for Computing Machinery	X	X	X	X						Apresenta o pytest-ranking, plugin para pytest que prioriza testes combinando três critérios: menor tempo de execução, falhas do último ciclo e proximidade de mudanças nos arquivos do commit. Avaliado em projetos Python open source no GitHub com pipelines de CI. Métricas: APFD, APFDc, runtime e taxa de identificação de falhas.	É o artigo central para proposta de pesquisa: ferramenta Python/pytest open source de priorização, avaliada em ambiente real de CI. Demonstra que a combinação dos três critérios supera cada um isoladamente. Limitação: não realiza seleção de testes, apenas reordena a suíte completa, o que não reduz o tempo total de execução.
Cheng, R., Wang, S., Jabbarvand, R., and Marinov, D. (2024). Revisiting Test-Case Prioritization on Long-Running Test Suites. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2024, pages 615–627, New York, NY, USA. Association for Computing Machinery	X	X	X	X						Estudo empírico em larga escala que avalia 59 técnicas de TCP em suítes com duração média de 6,5 horas. Propõe o dataset LRTS com 10 projetos da Apache Software Foundation. A técnica mais eficaz foi a combinação de histórico de falhas recentes e custo de execução. Métricas: APFD e APFDc.	Incluído por fornecer evidências empíricas em larga escala sobre eficácia de técnicas simples vs. complexas em CI. Confirma que heurísticas baseadas em histórico e runtime superam ML em cenários realistas. O dataset LRTS inclui projetos Python (TVM). Útil como baseline de comparação para o TCC.

Continua na próxima página

Tabela 1 – continuação da página anterior

Identificação do Trabalho	I1	I2	I3	I4	E2	E3	E5	E9	E10	Descrição	Avaliação
Torbunova, A., Strandberg, P. E., and Porres, I. (2024). Dynamic Test Case Prioritization in Industrial Test Result Datasets. In *Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024)*, AST '24, pages 154–158, New York, NY, USA. Association for Computing Machinery		X	X							Propõe algoritmo dinâmico de priorização baseado em probabilidade condicional, que ajusta scores de prioridade em tempo real com base no histórico de falhas e correlação entre testes. Ferramenta disponível no GitLab em Python. Avaliado em três datasets industriais (Westermo, Paint Control e ABB). Métrica: APFD.	Incluído por implementar a ferramenta em Python e por tratar priorização dinâmica em contexto de CI industrial. O mecanismo de ajuste em tempo real é relevante para cenários de suítes evolutivas. Limitação: avaliado em datasets proprietários de sistemas embarcados, sem aplicação direta em projetos Python open source.
Chen, Z., Chen, J., Wang, W., Zhou, J., Wang, M., Chen, X., Zhou, S., and Wang, J. (2023). Exploring Better Black-Box Test Case Prioritization via Log Analysis. *ACM Trans. Softw. Eng. Methodol.*, 32(3)							X			Propõe 7 técnicas de priorização black box (LogTCP) baseadas em análise de logs de execução, com representações por contagem, N-Gram e embeddings semânticos (FastText). Avaliado em 10 projetos Java open source. Métricas: APFD e RAUC-s.	Excluído por uso de PLN/ML na variante semântica (FastText), que é parte central do conjunto de técnicas proposto. As variantes sem ML são interessantes, mas o artigo como um todo é apresentado sob o guarda-chuva do LogTCP, que inclui a variante de embeddings.
Cheng, R., Zhang, L., Marinov, D., and Xu, T. (2021). Test-case prioritization for configuration testing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2021, pages 452–465, New York, NY, USA. Association for Computing Machinery					X					Propõe priorização de ctests (testes parametrizados por configurações) para detectar misconfigurações em projetos de computação em nuvem. Avaliado em HCommon, HDFS, HBase, ZooKeeper e Alluxio. Métricas: APMD e APMDc (adaptadas para configuração).	Excluído por foco exclusivo em testes de configuração. As métricas e o contexto são específicos para misconfigurações em ferramentas Java de computação em nuvem, sem aplicabilidade ao contexto de testes de regressão Python/pytest.
Elsner, D., Wuersching, R., Schnappinger, M., Pretschner, A., Graber, M., Dammer, R., and Reimer, S. (2022). Build system aware multi-language regression test selection in continuous integration. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, ICSE-SEIP '22, pages 87–96, New York, NY, USA. Association for Computing Machinery								X		Propõe seleção de testes em sistemas multilinguagem (Java + C/C++) ciente do sistema de build Maven, usando rastreamento de syscalls para detectar dependências entre DLLs nativas. Avaliado em sistema industrial da IVU Traffic Technologies (Windows). Métricas: proporção de testes selecionados, economia de tempo e segurança da seleção.	Excluído por ser método não aplicável à seleção ou priorização no contexto Python/pytest. O método é construído sobre Maven e dependências de DLL Windows, sem equivalente no ecossistema Python.
Chen, J., Lou, Y., Zhang, L., Zhou, J., Wang, X., Hao, D., and Zhang, L. (2018). Optimizing test prioritization via test distribution analysis. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2018, pages 656–667, New York, NY, USA. Association for Computing Machinery							X		X	Propõe o PTP (*Predictive Test Prioritization* ou Priorização de testes preditiva), que usa XGBoost para predizer quais testes têm maior probabilidade de falhar com base em distribuição histórica de falhas. Integrado ao Baidu CI. Avaliado em 50 projetos open source e 11 sistemas industriais. Métricas: APFD e APFDc.	Excluído pelo uso de ML supervisionado (XGBoost) como núcleo do método e pela avaliação sem qualquer consideração ao ambiente Python/pytest.

Continua na próxima página

Tabela 1 – continuação da página anterior

Identificação do Trabalho	I1	I2	I3	I4	E2	E3	E5	E9	E10	Descrição	Avaliação
Celik, A., Vasic, M., Milicevic, A., and Gligoric, M. (2017). Regression test selection across JVM boundaries. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2017, pages 809–820, New York, NY, USA. Association for Computing Machinery									X	Apresenta o RTSLinux, módulo do kernel Linux que rastreia dependências de arquivos via syscalls (chamadas do sistema) para seleção de testes além das fronteiras da JVM. Avaliado em 21 projetos Java open source. Métricas: proporção de testes selecionados e economia de tempo.	Excluído por inviabilidade de aplicação no ambiente Python/pytest. Em Python, o mecanismo de imports faz quase todos os testes dependerem de quase todos os arquivos, tornando a seleção imprecisa. O próprio artigo documenta essa limitação em seu estudo de caso Python.
Mendonça, W. D. F., Assunção, W. K. G., and Vergilio, S. R. (2023). Feature-oriented Test Case Selection during Evolution of Highly-Configurable Systems. In *Proceedings of the 27th ACM International Systems and Software Product Line Conference - Volume A*, SPLC '23, pages 76–86, New York, NY, USA. Association for Computing Machinery	X	X	X							Apresenta o FeaTestSel, ferramenta de seleção de testes orientada a features para sistemas altamente configuráveis em C/C++. Usa análise estática via Doxygen e grafos de dependência para mapear testes às features modificadas. Avaliado no Libssh. Métricas: redução de runtime e número de testes selecionados.	Incluído por propor seleção baseada em mudanças/features com redução de ≈50% no tempo total. O critério de seleção é conceitualmente adaptável, mas o mecanismo depende de diretivas de compilação C/C++ (#ifdef), sem equivalente direto em Python. Relevante como referência de seleção leve sem ML.
Meier, D., Mattis, T., and Hirschfeld, R. (2021). Toward Exploratory Understanding of Software using Test Suites. In *Companion Proceedings of the 5th International Conference on the Art, Science, and Engineering of Programming*, Programming '21, pages 60–67, New York, NY, USA. Association for Computing Machinery							X			Propõe priorização baseada em mutation testing combinada a predição de falhas via Decision Tree (Árvore de Decisão) e Random Forest, com ranqueamento por TF-IDF. Implementado como plugin para VS Code com backend em microsserviços. Avaliado no Flask com dataset de mutation testing gerado pelo Mutester.	Excluído por usar ML supervisionado (Decision Tree, Random Forest) e TF-IDF como métodos centrais de priorização. Apesar de usar Python e Flask como ambiente de avaliação, o método não é aplicável sem infraestrutura de ML.
Araújo, J., Araújo, J., Magalhães, C., Andrade, J., and Mota, A. (2017). Feasibility of using Source Code Changes on the Selection of Text-based Regression Test Cases. In *Proceedings of the 2nd Brazilian Symposium on Systematic and Automated Software Testing*, SAST '17, New York, NY, USA. Association for Computing Machinery	X	X								Apresenta o ATPCode, ferramenta de seleção de testes baseada em IR (*Information Retrieval* ou Extração de Informações) clássico com Apache Lucene. Extrai keywords de mudanças no código-fonte via regex CamelCase e busca testes relacionados. Avaliado em sistema industrial da Motorola Mobility. Métrica: cobertura de código obtida pelos testes selecionados.	Incluído como referência de seleção baseada em mudanças de código sem ML. O critério de seleção por keywords de mudança é conceitualmente adaptável ao contexto Python/pytest. Avaliado em testes semi-manuais em linguagem natural, o que limita a transferência direta do método.

Continua na próxima página

Tabela 1 – continuação da página anterior

Identificação do Trabalho	I1	I2	I3	I4	E2	E3	E5	E9	E10	Descrição	Avaliação
Elsner, D., Hauer, F., Pretschner, A., and Reimer, S. (2021). Empirically evaluating readily available information for regression test optimization in continuous integration. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2021, pages 491–504, New York, NY, USA. Association for Computing Machinery	X	X	X	X						ropõe e avalia as técnicas CI-RTP/S (Seleção e priorização de testes de regressão baseada em CI), que usam apenas metadados de CI e Git (sem acesso ao código-fonte) para priorização e seleção de testes em pipelines de CI. Avaliado em 23 projetos (3 industriais e 20 open source). Métricas: APFDc para priorização e economia de tempo para seleção.	Incluído por demonstrar que heurísticas simples com metadados de CI alcançam redução média de 84% no tempo de execução mantendo 90% de detecção de falhas. Evidência direta de que ML não é necessário para resultados expressivos em CI. Relevante para fundamentar a escolha de critérios simples no plugin proposto.
Mendonça, W. D. F., Vergilio, S. R., Michelon, G. K., Egyed, A., and Assunção, W. K. G. (2022). Test2Feature: feature-based test traceability tool for highly configurable software. In *Proceedings of the 26th ACM International Systems and Software Product Line Conference - Volume B*, SPLC '22, pages 62–65, New York, NY, USA. Association for Computing Machinery						X		X		Apresenta o Test2Feature, ferramenta de rastreabilidade estática entre testes e features em sistemas C/C++ com diretivas de pré-processador. Gera três tipos de relatórios de rastreabilidade via Doxygen e grafos de dependência. Avaliado no SQLite.	Excluído por dois critérios: foco na geração de rastreabilidade e não em seleção ou priorização, e por ser restrito a sistemas C/C++ com diretivas #ifdef, sem aplicabilidade ao ambiente Python/pytest.
Hu, Y., Ishibe, H., Dai, F., Yamazaki, T., and Chiba, S. (2024). Bugfox: A Trace-Based Analyzer for Localizing the Cause of Software Regression in JavaScript. In *Proceedings of the 17th ACM SIGPLAN International Conference on Software Language Engineering*, SLE '24, pages 224–233, New York, NY, USA. Association for Computing Machinery								X	X	Apresenta o Bugfox, analisador de rastros de execução JavaScript para localizar a causa de regressões, usando as heurísticas para identificar o ponto de desvio entre versões. Avaliado em 12 regressões do benchmark BugsJS. Métricas: acurácia, runtime e overhead de memória.	Excluído por dois critérios: o método é de localização de causa de falha (*fault localization*), não de seleção ou priorização de testes, e é restrito ao ecossistema JavaScript, sem aplicabilidade ao ambiente Python/pytest.